Enabling Resilience in the Internet of Things

Kyle Benson
Dept. of Computer Science, University of California, Irvine, USA
Email: kebenson@ics.uci.edu

***Abstract*—This dissertation explores issues of resilience in pervasive Internet of Things (IoT) deployments. It discusses facilitating the classic *observe-analyze-adapt* loop by addressing problems with devices, networks, and applications. We discuss our experiences with two projects: the Safe Community Awareness and Alerting Network, which leverages pervasive devices to extend a smarter safer home to residents, and the Community Seismic Network, which creates a massive network of low-cost sensors for detecting earthquakes. We propose a middleware framework for improving resilient communications between devices and services, facilitating a scalable resilient multi-protocol data exchange, and supporting more resilient distributed applications.**

I. Introduction

With the increasing pervasiveness of computers and connected devices in our daily lives, the Internet of Things (IoT) concept promises a myriad of possible applications ranging from manipulating devices in our homes to large-scale automation of industries and public utilities. However, as these deployments grow in scale and scope, so does our dependence on their proper performance. This opens several questions in how to build such deployments and their associated applications in a more resilient manner. Oftentimes these devices are inexpensive and disposable, resulting in frequent failures or incorrect actions. The sheer number of these devices introduces concerns of global scalability on an unprecedented level, requiring novel techniques for addressing spikes in demand, network congestion, etc. In addition, these devices come from different vendors and serve different purposes, resulting in a high degree of heterogeneity, which makes addressing resilience concerns across all devices even more difficult. In order to compose new services from these heterogeneous devices, we must address interoperability concerns and enable true extensibility, which further complicates resilience by allowing combinatorially more possible arrangements. Different deployments, domains, and application scenarios may have differing resilience issues that must be addressed (e.g. factory automation vs. home health monitoring). Lastly, applications and services should be resilient to software faults, incorrect information, network outages, and all of the aforementioned device issues.

We identified these key areas based on our previous experiences, described in Section II, with cyber-physical systems (CPS) and IoT deployments, including two major projects we plan to use as testbeds in our work. We propose in Section III how we will address some of these issues through a resilient middleware that facilitates *resilient communications, data exchange, and service execution*.

II. Our Experiences and Testbeds

To gain experience in IoT deployments we have worked with several systems, two of which we discuss here. We are deploying a local instance of the Community Seismic Network (CSN) [5], which uses a multitude of inexpensive accelerometers to rapidly detect earthquakes, on the UCI campus and have discussed the Caltech team's experiences with this system. One key observation is that reliance on a specific cloud platform makes the system fragile in the event of an incredibly powerful earthquake that may cause massive network outages. Without being able to contact the data center in Arizona, important sensor readings cannot be uploaded for use by emergency response personnel. We plan to use this local CSN deployment as a testbed for experimenting with managing, storing, and processing large volumes of streaming sensor data, particularly with spatiotemporal components. Furthermore, we are exploring the possibility of implementing a distributed algorithm for earthquake-detection run by the sensor devices themselves. This would allow the system to continue functioning, albeit with a potentially degraded quality of service, even in the event of losing all long-haul connectivity to data centers inside or outside the impacted region.

In response to the SmartAmerica Challenge [4], the multi-organization Safe Community Awareness and Alerting Network (SCALE) team convened to envision, design, build, and demonstrate a system for addressing community and resident safety by leveraging IoT devices and services. We implemented SCALE as the event-driven middleware platform depicted in Figure 1: devices upload sensed events to a cloud data exchange and the analytics service retrieves these events, scanning for possible emergencies and sending residents alerts to confirm or reject the emergency. SCALE was designed and implemented in the course of four months and so was predominately an exercise in service composition and device interoperability as we integrated devices from our various partners all in the same system. However, we learned several important lessons regarding resilience from SCALE that are helping guide our research.

First, we witnessed firsthand the importance of addressing device failures as we used cheap off-the-shelf sensors with short lifespans and sometimes inaccurate readings. Second, we encountered spotty network connections and determined that our sensor clients needed more intelligence to choose from one of their several network adapters to improve chances of data delivery. Third, our use of MQTT [1], a lightweight publish-subscribe-style protocol, for the data exchange posed limitations on expressiveness of event subscriptions. Its hierarchical topic schema and lack of range queries made subscribing only to specific sensor readings difficult. Furthermore, MQTT's centralized broker architecture introduced a point of failure and rendered the system vulnerable to outages (e.g. server crash,

978-1-4799-8425-1/15/$31.00 ©2015 IEEE

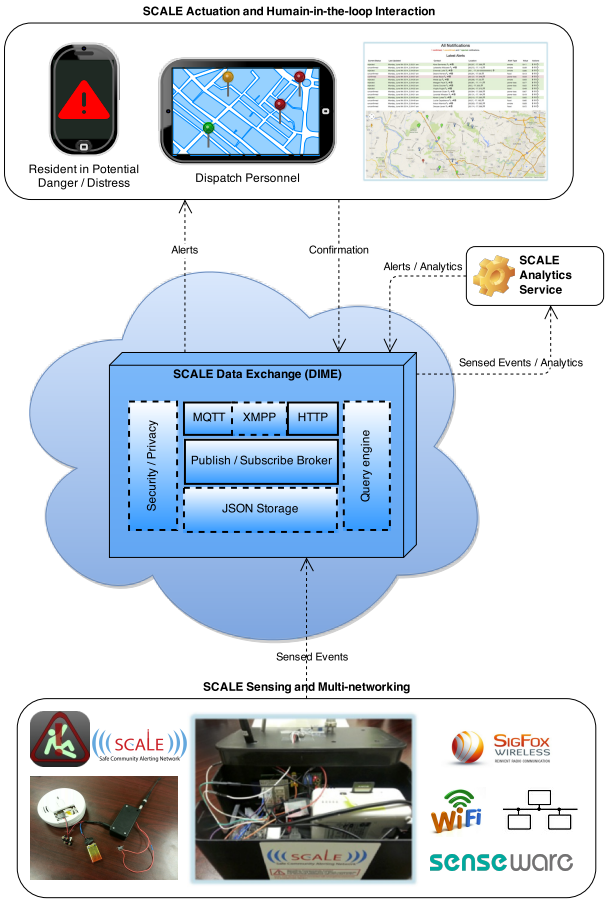


Fig. 1. An overview of the SCALE system: clients send data to a cloud data exchange where it can be retrieved by the analytics service for event-detection and the dashboard for visualization.

network faults, etc.). Fourth, we found ourselves reinventing the wheel when designing the application software (analytics service). For example, we had to disambiguate events by correlating individual readings with the same perceived real-world event. This was exacerbated by running multiple server instances, which brought its own challenges, and so a framework for effectively handling data in a distributed application would have greatly simplified the system development.

The well-received SCALE project is currently continuing into a second phase targeting the Global City Teams Challenge [7]. The team plans to deploy SCALE devices from various partners in a senior living facility to experiment with various personal and community safety-oriented applications. We will also expand the deployment of SCALE devices on the UCI campus, merging them with the CSN deployment, to conduct more experimental tests without disrupting those in active use by real people. These tests will include an *emergency mode* in which SCALE clients switch to a low-power mode and activate a wireless ad-hoc network to continue running emergency applications during power outages.

III. Current and Proposed Research

This section discusses three components of our proposed resilient middleware for the IoT.

A. Resilient Communications

As communicating sensor data and other messages are a centerpiece of IoT applications, we must enable resilient machine-to-machine communications in a manner that application programmers need not concern themselves with. During our discussions with the CSN team, we realized that a massive earthquake and the resulting power outages could render the system unusable due to its reliance on communicating data from the sensors to a centralized cloud server. This motivated our ongoing work in establishing a location-aware resilient overlay network for improving sensor data delivery during such a scenario. The key idea is to use overlay peers to enable source routing and trick the underlying routing infrastructure to make use of available routes that it is not yet aware of, as shown in Figure 2. This technique was originally proposed in [8] but did not consider geo-location information in choosing candidate peers.

In our previous work [9], we proposed the Geographically-Correlated Resilient Overlay Network (GeoCRON) system and showed the efficacy of this approach through simulations using simple heuristics. These heuristics exploited only knowledge of the overlay peers and their locations in choosing alternate routes. We explored various choices of peers based on the angle they make with the source and destination (routing along an indirect trajectory), their distance from the source, and the geographic region they occupy. This work assumed the peers knew nothing about the underlying network topology. However, this resulted in minimal improvement and only for certain topologies, and so we are now currently exploring techniques for choosing peers based on knowledge of the underlying network topology and the locations of the routers and links therein. Some previous work [6], [10] looks at choosing geo-diverse paths along which to send information, thereby increasing the chances of successful delivery and thus communications resilience. We hope to borrow some of these techniques and adapt them to choosing overlay peers rather than directly specifying the route for a packet to traverse through the physical routers.

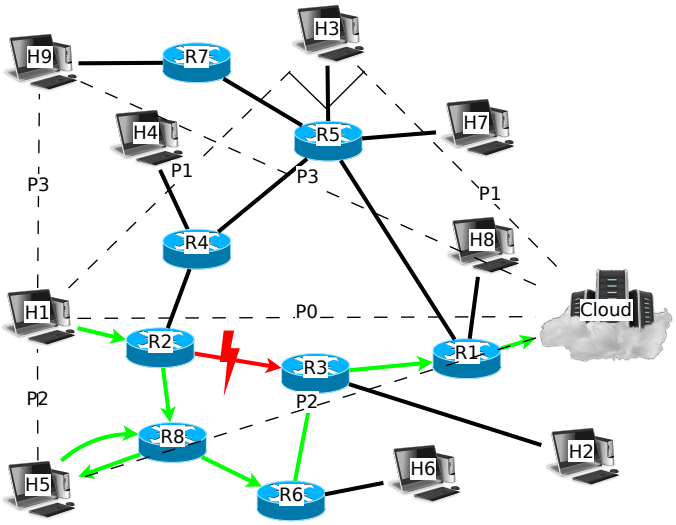


Fig. 2. Geo-correlated resilient overlay networks exploit location information of nodes to improve data delivery to a cloud server.

In addition to Internet overlays, we are also exploring more

local forms of resilient communications. For example, we are adding wireless ad-hoc networking capabilities to the SCALE client devices so that they may still communicate in the event of a failed access point, possibly due to a power failure. This will allow us to investigate efficiently transmitting data along multiple hops to upload it via a different medium (e.g. satellite) as well as strategies such as mobile message ferrying and choosing between multiple adapters using multi-network state information.

B. Resilient Data Exchange

To facilitate exchanging IoT data (sensed events, analytics, alerts, etc.) without requiring the end points to directly address each other, we propose a conceptual data exchange that we envision as an open communications hub for IoT. It should contain middleware abstractions for enabling hosts to connect via multiple protocols, discover available data, issue historical queries, and filter streaming subscriptions. The implementation must also support a large network of brokers to improve scalability and eliminate central points of failure. Rather than a centralized client-server model, we envision a highly distributed hybrid approach, leveraging cloud-based data centers as well as local cloudlets and individual peer devices.

To implement such an exchange, we are currently considering the use of more distributed pub-sub protocols such as DDS [2]. We are also researching services that bridge multiple protocols, such as Eclipse Ponte [3]. To deploy such a service as a global data exchange, we will investigate techniques for efficiently distributing subscriptions and replicating data among a network of brokers, taking into account scalability but also resilience to failures and traffic spikes. As a first step towards such a system, we will add some data exchange features to GeoCRON. As communications and data exchange are closely related, this will allow us to take into account location information and network topologies to improve resilient data exchange with other devices. We plan to explore such problems as the placement of broker replicas, aggregating sensor data at local peers to alleviate congestion, and caching information on nearby peers and local servers both to improve the chances of recovering that data in the event of a disaster and also to enable local event-detection on the nodes holding these caches.

C. Resilient Application Execution

To enable resilient deployment and execution of IoT applications, we propose using a distributed execution environment for logic and analytics. While more complicated to implement and manage than the traditional centralized approach, such an architecture could mitigate the effects of component failures on an overall system by providing redundant channels to accomplish business logic. Our first investigation into this concept is the implementation of a distributed earthquake-detection algorithm for our CSN testbed. Built on top of GeoCRON for improved resilience, we will explore how to fine-tune overlay peer choices and local exchange of sensor data to more quickly and reliably detect, characterize, and issue alerts about earthquakes in a region. This will help inform about some of the issues inherent with this approach, including necessary primitives to build the application and how to coordinate such real-time detection efforts.

In the future, we also plan to investigate a mobile agent-based approach in which application logic is decomposed and distributed across multiple devices and even data centers. Consider an IoT application workflow as being composed of a series of steps, or *nodes*, through which data flows, starting with a sensed event or user command. By considering these *nodes* as mobile agents that consume some data and forward the answer to the next *node* in line, we can realize a framework for developing application workflows that naturally maps to a dynamically adaptive distributed execution environment. Applications can be considered as operator graphs and deployed to entire networks, rather than individual devices, by developing a plan for ideally placing *nodes*, taking into account latency, CPU, storage, location, reliability, security constraints, etc. These agents can then migrate between devices for scalability, replicate for reliability, or stay within system boundaries for policy compliance (e.g. security and privacy). This platform could leverage the aforementioned resilient data exchange by migrating *nodes* and moving or merging data streams together to exploit locality and further improve the overall system efficiency. We can also easily write new applications by reusing *nodes* from previous ones or even simply modify individual *nodes* to hook into a step in a pipeline.

REFERENCES

[1] MQTT. `http://mqtt.org`.

[2] OMG Data Distribution Portal. `http://portals.omg.org/dds/`.

[3] Ponte - Bringing Things to REST Developers. `http://eclipse.org/ponte/`.

[4] SmartAmerica. `http://smartamerica.org/`.

[5] Community Seismic Network. `http://www.communityseismicnetwork.org/`, Jul 2012.

[6] Path diversification for future internet end-to-end resilience and survivability. *Telecommunication Systems*, 56(1):49–67, Aug. 2013.

[7] Global City Teams Challenge. `https://us-ignite.org/globalcityteams/`, 2014.

[8] D. Andersen, H. Balakrishnan, F. Kaashoek, and R. Morris. Resilient overlay networks. In *Proceedings of the eighteenth ACM symposium on Operating systems principles*, SOSP '01, pages 131–145, New York, NY, USA, 2001. ACM.

[9] K. E. Benson and N. Venkatasubramanian. Improving Sensor Data Delivery During Disaster Scenarios with Resilient Overlay Networks. In *Third International Workshop on Pervasive Networks for Emergency Management 2013 (PerNEM 2013)*, 2013.

[10] K. Kim and N. Venkatasubramanian. In *2010 IEEE Global Telecommunications Conference GLOBECOM 2010*.